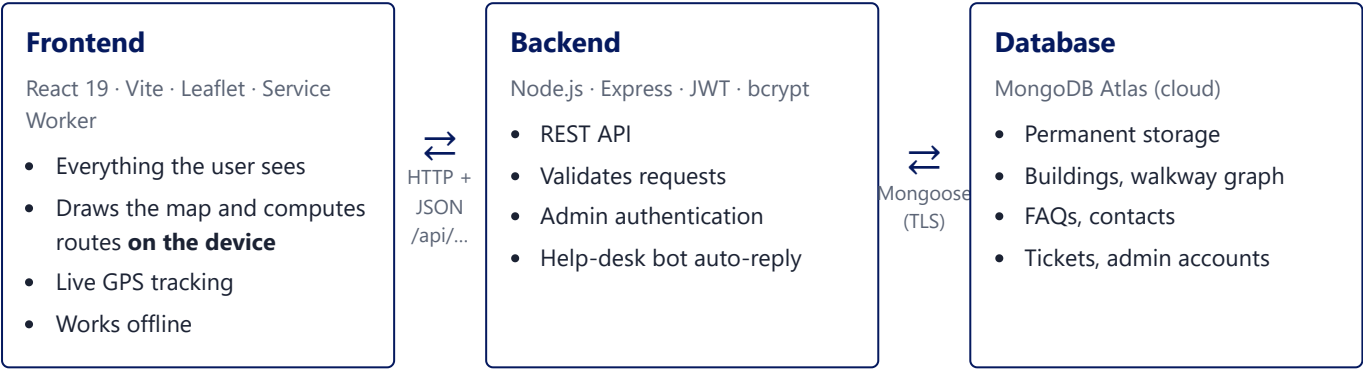


GCTU Campus Navigator

System guide for the project defense: how the frontend, backend, database and mapping work

1 The system at a glance

The Campus Navigator is a **Progressive Web App (PWA)** that helps students and visitors find their way around GCTU's Tesano campus. It has three parts, each with one job:



The one design decision to explain first

Navigation runs **entirely in the browser**. The campus map data (buildings and walkway graph) is bundled with the app, and the shortest path is calculated on the phone itself. That is why directions still work with no internet. The backend and database handle what has to be **shared or persistent**: help-desk tickets, admin login, FAQs and contacts. They also hold a copy of the campus data, available through the API.

Layer	Key technologies	Main files
Frontend	React 19, Vite 6, Leaflet 1.9 + react-leaflet 5, lucide icons, Tailwind/CSS	src/App.jsx , src/components/* , src/data/* , public/sw.js
Backend	Node.js, Express 4, jsonwebtoken, bcryptjs, cors, dotenv	server/index.js , server/routes/* , server/middleware/auth.js
Database	MongoDB Atlas, Mongoose 8 (schemas and models)	server/models/* , server/db.js , server/seed/seed.js
Testing	Vitest, React Testing Library, jsdom	34 UI tests + 14 live API/database tests

2 How the frontend works

Structure

`main.jsx` mounts the root `App` component, which renders three tabs. The code is organised by responsibility: **components** (UI, split into small pieces per feature), **hooks** (reusable stateful logic such as `useLiveGps` , `useWalkSimulation` and `useAdminSession`), **lib** (pure logic with no React: Dijkstra, Haversine, the API client) and **data** (the campus graph and help-desk content, shared with the database seed).

Tab / component	What it does
Campus · CampusHome	Hero slideshow of buildings (auto-advances every 5 s), campus stats, and a grid of building cards. Every card has a <i>Get Directions</i> button.

Navigate · <code>NavigationPanel</code>	Route planner (start and destination), Leaflet map, shortest-path route line, turn-by-turn directions, live GPS and a Walk Demo simulation. See Section 3.
Help Desk · <code>HelpDesk</code>	FAQ accordion, support hotlines, a form to submit a help request, and an admin sign-in panel for managing tickets.

Key behaviours to know

- **Tabs are always mounted.** Switching tabs only toggles a CSS class (`tab-panel--active`), so the map is not destroyed and rebuilt every time. Map state and any running GPS session survive a tab change.
- **Cross-tab communication.** Pressing *Get Directions* on the Campus tab calls `handleNavigateTo(name)` in `App`. That stores a `presetDestination` and switches to Navigate, where `NavigationPanel` finds the building and sets it as the destination (start = Main Gate).
- **State management** uses React hooks only (`useState` , `useEffect` , `useRef`). The app is small enough that no Redux-style store is needed.
- **Theme.** Light or dark mode is saved in `localStorage` (defaulting to the OS preference), applied as a `dark` class on `<body>`, and the map tiles are colour-inverted with a CSS filter.
- **Talking to the API.** The Help Desk uses `fetch()` against `/api/...`. In development, Vite proxies `/api` to Express on port 5000. Every response is checked: if the API is down or returns an error, the Help Desk falls back to a bundled copy of the FAQs and contacts instead of crashing.
- **Admin session.** After sign-in, the JWT is stored in `localStorage` and sent as `Authorization: Bearer <token>`. If the server answers 401 (the token expired), the UI signs the admin out and asks them to sign in again.

PWA and offline mode

A **service worker** (`public/sw.js`) sits between the app and the network. It uses a different caching strategy for each kind of request:

Request type	Strategy	Why
App shell (HTML, JS, CSS)	Cache first, refresh in background	Opens instantly and works offline.
Map tiles (OpenStreetMap)	Cache on first view	Areas you have viewed stay visible offline. An uncached tile is replaced by a "Map Offline" placeholder.
API data (<code>/api/...</code>)	Network first, cache as fallback	Tickets change, so fresh data wins. Cached FAQs are used only when offline. Admin responses are never cached.

The app also shows an *Install* banner (the `beforeinstallprompt` event) and an offline banner driven by the browser's `online` / `offline` events.

3 How the mapping and navigation work

3.1 The base map

The map is drawn with **Leaflet** through **react-leaflet**. The background imagery is **OpenStreetMap tiles** (256×256 px images at `.../{z}/{x}/{y}.png`), which are free and need no API key. OpenStreetMap has no dark style, so in dark mode a CSS filter inverts the tile colours. The map opens at the campus centroid `[5.5966, -0.2234]`, zoom 18. On top of the tiles the app draws its own layers: building markers (emoji icons), the Main Gate, start and destination pins, the blue route line with a shadow, a distance badge, and the GPS marker with its accuracy circle.

3.2 The campus as a graph

Walkways are modelled as a **weighted, undirected graph** in `src/data/buildings.js`:

- **15 nodes**: 10 buildings, plus the Main Gate and 4 walkway **junctions** (points where paths meet). Each node has a real GPS coordinate.
- **26 edges**: the walkable paths between nodes. **Undirected** means you can walk each path both ways.
- **Edge weight** = the real ground distance between the two ends, in metres.

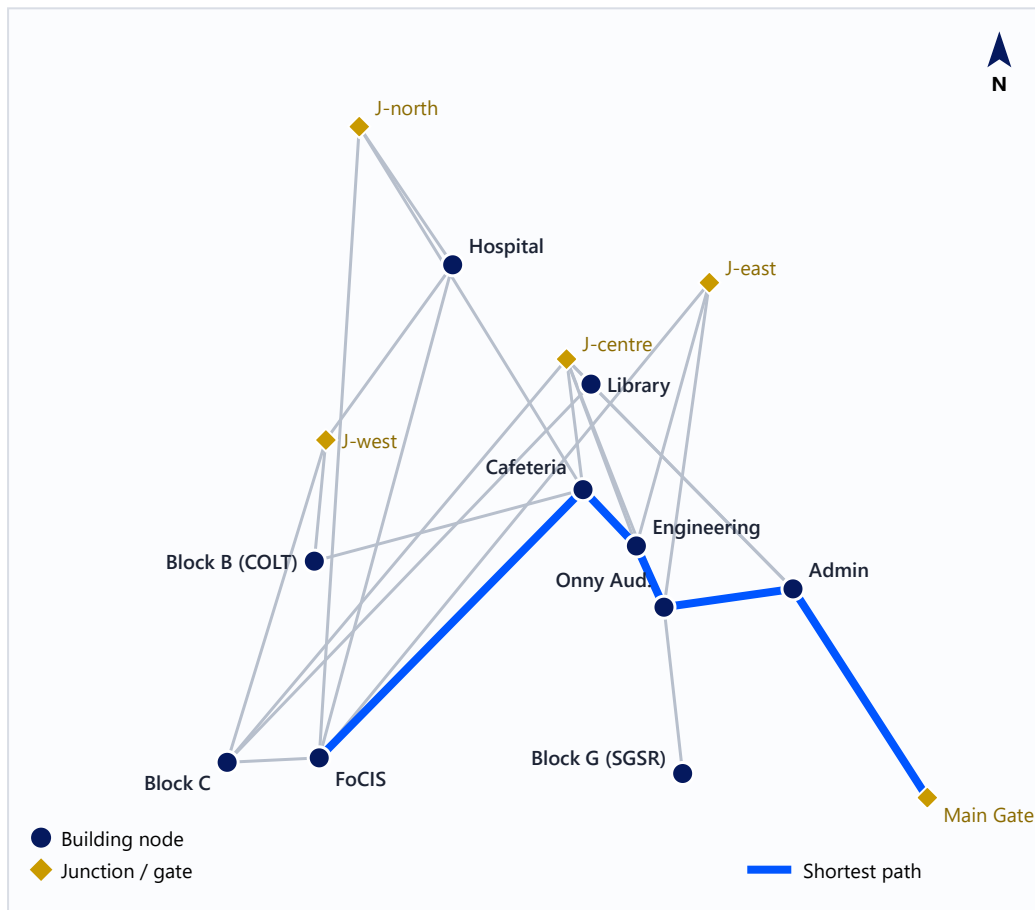


Figure 1: the campus walkway graph drawn from the real coordinates. The highlighted path is the shortest route from the Main Gate to FoCIS (223.8 m).

3.3 Distance: the Haversine formula

Coordinates are points on a sphere, so straight-line (Pythagoras) distance on lat/lng would be wrong. `getDistance()` uses the **Haversine formula** for the great-circle distance:

$$a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2)$$

$$d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \quad \phi = \text{latitude}, \lambda = \text{longitude (radians)}, R = 6,371,000 \text{ m}$$

3.4 Shortest path: Dijkstra's algorithm

`findDijkstraPath(start, end)` finds the route with the smallest total walking distance:

1. Set every node's distance to ∞ , except the start (0). Put all nodes in an *unvisited* set.
2. Take the unvisited node with the smallest distance. Stop when it is the destination.
3. For each neighbour, compute *current distance* + *edge length*. If that is shorter than the neighbour's recorded distance, update it and remember `previous[neighbour] = current`.
4. Remove the current node from *unvisited* and repeat.
5. Rebuild the path by following `previous[]` backwards from the destination.

Worked example: Main Gate → Faculty of Computing (FoCIS)

Gate → Admin (61.8 m) → Florence Onny Auditorium (32.4 m) → Engineering (16.7 m) → Cafeteria (19.3 m) → FoCIS (93.6 m) = **223.8 m**.

Complexity: $O(V^2)$ with a simple scan for the minimum. With $V = 15$ this takes well under a millisecond, so a priority queue is unnecessary. Dijkstra is guaranteed to find the optimum because every edge weight (a distance) is non-negative.

3.5 Turn-by-turn directions

`generateTurnByTurn()` turns the path into instructions. At each intermediate node it compares the **bearing** of the incoming segment with the bearing of the outgoing one (computed with `atan2`). The difference, normalised to $-180^\circ \dots +180^\circ$, decides the instruction:

Angle change	< 22°	22–65°	65–125°	≥ 125°
Instruction	Continue straight	Bear slightly right / left	Turn right / left	Sharp right / left

A positive angle means a turn to the right, a negative one to the left. The first step is always "Depart from ...", and each step shows its distance in metres.

3.6 Live GPS tracking

- Uses the browser **Geolocation API**: `navigator.geolocation.watchPosition()` with `enableHighAccuracy`. It needs HTTPS or localhost; the app checks for this and explains if it is missing.
- Every GPS fix updates the blue marker and an **accuracy circle** (radius = the reported accuracy in metres).
- **Snapping to the route.** Over a campus-sized area, lat/lng can be treated as a flat grid (1° latitude $\approx 111,320$ m). The fix is projected onto the nearest route segment, which gives three values: *off-route distance*, *distance travelled* and *distance remaining*. The current segment decides which direction step is highlighted.
- If the user is more than 300 m from the route, the app warns that they are not on campus.
- Only "permission denied" stops tracking. Timeouts and "position unavailable" are temporary, so tracking carries on.
- **Camera behaviour.** With no GPS, the map zooms to fit the whole route. While tracking, it centres once and then pans only when the marker leaves the inner 70% of the screen, so the marker is seen moving.

3.7 Walk Demo (simulation)

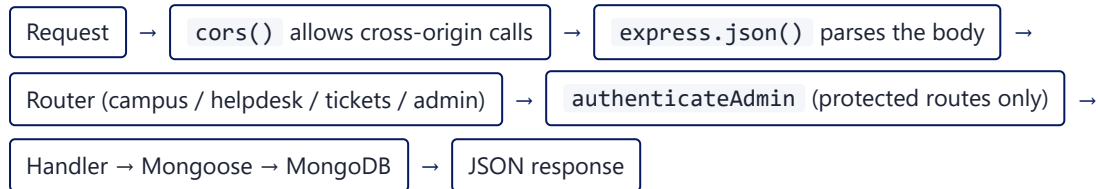
For a demonstration away from campus, *Walk Demo* moves a simulated GPS marker along the route at walking pace (1.4 m/s, sped up 5×). It updates every 100 ms and **interpolates** the position along each segment, so the

motion is smooth rather than jumping from node to node. On arrival it announces the destination with the browser's speech synthesis.

4 How the backend API works

Request pipeline

`server/index.js` creates an Express app. Every request passes through the same chain:



Each handler is wrapped in `try/catch` and returns JSON with the right HTTP status. Unknown routes return **404**, malformed JSON returns **400**, and anything unexpected returns **500**, always as JSON. The server only starts listening **after** the database connection succeeds.

Endpoints

Method	Endpoint	Purpose	Access
GET	<code>/api/health</code>	Liveness check	public
GET	<code>/api/buildings</code>	All buildings, sorted by name	public
GET	<code>/api/buildings/search?q=COLT</code>	Full-text search on names, aliases and rooms, ranked by relevance	public
GET	<code>/api/buildings/:id</code>	One building (404 if not found)	public
GET	<code>/api/graph</code>	Walkway graph as <code>{ nodes, edges }</code>	public
GET	<code>/api/faqs</code> · <code>/api/faqs/search?q=</code>	FAQ list · full-text FAQ search	public
GET	<code>/api/contacts</code>	Support hotlines	public
POST	<code>/api/tickets</code>	Submit a help request (201 Created)	public
POST	<code>/api/admin/login</code>	Exchange username and password for a JWT	public
GET	<code>/api/tickets</code>	All tickets, newest first	admin JWT
PATCH	<code>/api/tickets/:ticketId</code>	Update a ticket's status or reply	admin JWT

Ticket lifecycle (with the automatic bot reply)

1. **Validate:** `name`, `subject` and `message` are required, otherwise the API returns **400**.
2. **Create:** the API generates an ID like `TKT-4821` (retrying if the ID is already taken), saves the ticket with `status: "open"`, and answers **201**.
3. **Bot reply:** 2.5 s later the server scans the subject and message for keywords (*wifi / internet, clinic / sick, portal / exam, fees / bank*), writes a matching answer and sets `status: "replied"`. It only touches tickets that are still open, so a human admin's reply is never overwritten.

4. **Admin review:** a signed-in admin sees all tickets and can mark any of them as replied.

Authentication and security

- **Password storage:** admin passwords are hashed with **bcrypt** (10 salt rounds) and never stored in plain text. On the very first login, if no admin exists, one is created from the `.env` credentials.
- **Login:** `bcrypt.compare()` checks the password. On success the server signs a **JWT** containing the admin's id and username, valid for **8 hours**. Wrong credentials always return the same *"Invalid credentials"* message, so an attacker cannot tell whether a username exists.
- **Protected routes:** the `authenticateAdmin` middleware reads the `Bearer` token and verifies its signature and expiry. A missing, forged or expired token gets **401**.
- **Secrets** (database URI, JWT secret, admin credentials) live in `.env`, which is excluded from Git.

5 What the database does

The system uses **MongoDB Atlas**, a cloud-hosted NoSQL **document database**, accessed through **Mongoose**. Mongoose adds schemas (field types, required fields, defaults, allowed values) on top of MongoDB's flexible JSON-like documents.

Collection	Key fields	Role / indexes
buildings	<code>id</code> , <code>name</code> , <code>shortName</code> , <code>category</code> , <code>lat</code> , <code>lng</code> , <code>desc</code> , <code>facts[]</code> , <code>aliases[]</code> , <code>rooms[{name, floor, capacity}]</code>	Campus directory. Text index on <code>name</code> , <code>shortName</code> , <code>aliases</code> and room names (so a search for "Room G6" or "COLT" works). <code>id</code> is unique.
graphnodes	<code>id</code> , <code>name</code> , <code>lat</code> , <code>lng</code> , <code>type</code> ("building" or "junction")	Vertices of the walkway graph
graphedges	<code>from</code> , <code>to</code>	Walkable paths between nodes (undirected)
faqs	<code>faqId</code> , <code>category</code> , <code>question</code> , <code>answer</code>	Knowledge base. Text index for FAQ search.
contacts	<code>dept</code> , <code>phone</code>	Support hotlines
tickets	<code>ticketId</code> , <code>name</code> , <code>faculty</code> , <code>subject</code> , <code>message</code> , <code>status</code> , <code>reply</code> , <code>date</code>	Help requests. <code>ticketId</code> is unique. <code>status</code> must be "open" or "replied".
admins	<code>username</code> , <code>password</code> (bcrypt hash)	Staff accounts. <code>username</code> is unique.

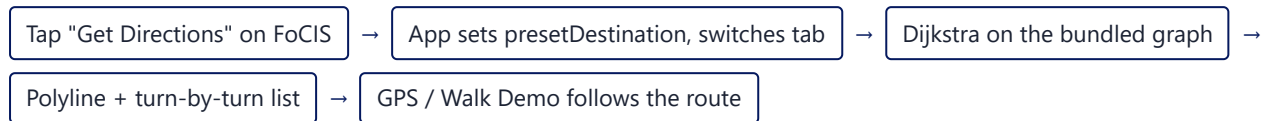
All documents get automatic `createdAt` / `updatedAt` timestamps. Tickets are listed newest first by `createdAt`.

- **Seeding:** `npm run seed` clears the collections and loads them from the same data files the frontend bundles (`src/data/buildings.js`, `src/data/helpdesk.js`). The database and the offline app therefore always hold identical campus data, maintained in one place.
- **Why MongoDB:** the data is document-shaped. A building naturally contains lists of aliases, facts and rooms, which would need several joined tables in SQL. Its built-in text indexes provide search, and Atlas offers free, managed cloud hosting.
- **Connection resilience:** Atlas uses a `mongodb+srv://` address, which needs DNS SRV and TXT lookups. Some networks (mobile hotspots, campus DNS) block TXT lookups. `db.js` detects this, resolves the cluster hosts

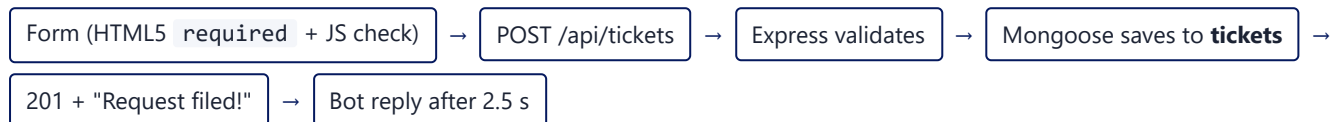
itself, asks the cluster for its replica-set name, and connects with a plain host list instead.

6 End-to-end flows

A: Student gets directions (no server involved)



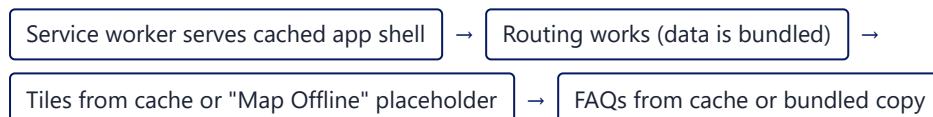
B: Student submits a help request



C: Admin manages tickets



D: Opening the app with no internet



7 Likely defense questions

Why Dijkstra and not A*?

With only 15 nodes, Dijkstra is instant and always finds the shortest path. A* adds a heuristic that only pays off on large graphs such as city road networks. Haversine distance would make an admissible heuristic if the graph grew.

Why compute routes on the phone instead of the server?

It works offline, answers instantly with no network round-trip, and adds no server load. The campus graph is tiny (a few KB), so shipping it with the app costs nothing.

How accurate is the GPS?

Phone GPS is typically accurate to 3–10 m outdoors and worse beside buildings. The app shows the reported accuracy as a circle and snaps the position to the nearest point on the route, so small errors don't break the guidance.

How is the admin area secured?

Passwords are stored as bcrypt hashes, sessions use a signed JWT that expires after 8 hours, and every admin endpoint verifies the token server-side. Hiding the admin panel in the UI is only for convenience: the server itself refuses requests without a valid token.

What happens if the database is down?

The server does not start without a database connection. In the app, navigation is unaffected (it runs locally), the Help Desk falls back to its bundled FAQs and contacts, and submitting a ticket shows an error message instead of failing silently.

How was the system tested?

With Vitest. There are 34 UI tests (map rendering, route distance, redrawing the route, the walk demo and live GPS logic, help-desk form validation, admin login, offline service worker) and 14 integration tests that call the real Express API and check the results in MongoDB Atlas (ticket persistence, bcrypt storage, JWT access control, rejection of expired or forged tokens).

What are the limitations and future work?

Node coordinates were placed by hand from Google Maps, so the graph is only as accurate as that mapping. There is no indoor (floor-level) routing yet, and the bot reply works on keywords rather than natural-language understanding. Possible next steps: an admin screen for editing the map, indoor floor plans, and push notifications when a ticket is answered.